P4 type inference

Canonical name: type retrieval. The implementation module remains `TypeInference`; the canonical name is `TypeRetrieval`. It retrieves and elaborates types against the signature table; it is not a type checker. Type checking is a later layer. See `docs/irs.typ`.

The target

P4 is the type-elaboration layer between a Scale expression and the signature table. It does not invent types by observing execution. It collects constraints from the expression, solves them against the declared word signatures, and returns either a typed result or a named diagnostic. The signature and effect table remains the authority.

P4a is the host-side constraint and unification core. It owns type variables, function types, schemes, substitutions, effects, and constraints. It uses a first-order unifier with an occurs-check. It has no parser dependency, no image dependency, and no runtime-term dependency.

What a Scale author can do

With the P4b contract, a Scale author writes ordinary function bodies without annotating every local. The elaborator checks them against the word-signature table:

1. call a known word and receive its declared result type and effects;
2. write a lambda or `logic` body whose parameters are inferred from use;
3. pass a function as a value, apply it, and let application determine its type;
4. partially apply a word or function, including explicit _ argument holes;
5. write `if` branches that are unified to one result type;
6. write a pipeline as the corresponding call;
7. bind a local with `logic` and use it polymorphically where generalization is sound.

The author still writes the meaning. P4 infers only the static type and effects; it does not invent a word, a signature, or a runtime behavior.

What the author must provide or avoid

- There is no user-facing signature syntax. The author writes the body; primitive word types come from an internal registry maintained with the implementation. An unknown or ambiguous word is still a named diagnostic.
- The internal signature and effect facts must be accurate. P4b uses exact effect sets; effect rows and subtyping are later work.
- A hole is an omitted call argument. It is not a declaration parameter hole and must not appear where a value is required.
- Pattern matching and the declaration forms are outside P4b; they are refused with a named diagnostic rather than guessed.
- Do not rely on runtime tensor tags or stack shape to choose a type. Those are runtime checks, not inference sources.
- Do not expect annotations to override the registry. There are no annotations in the first surface; if one is added later it is checked, not trusted.

Internal primitive registry

The no-signature design does not mean the checker has no type facts. It means those facts are not source syntax:

- Primitive words such as `add`, `subtract`, `multiply`, `divide`, `lessThan`, `greaterThan`, and `equals` carry their signatures in `components/prelude/evaluator/signatures.fs`. They are implementation metadata, versioned with the evaluator.

- F# package bindings get their types from FCS read-off at the package boundary.
- Scale `logic` definitions are inferred from their bodies with let generalization; their bodies constrain the parameter and result variables.
- An empty or ambiguous expression is a diagnostic. The author changes the use, not a declaration, to disambiguate it.

The same file carries `classification`, the single word index: every explicit case in `Evaluator.executeBuiltinWord` is either an `Expression` (a callable signature) or a `NotInferred` word with its reason. `primitiveTable` is derived from that list, not maintained beside it, and `just docs-check` fails if the dispatcher and the index diverge. The dispatcher remains the execution authority; the index is the inference-coverage authority.

The limits are explicit: polymorphic recursion is not expressible without annotations, multiple dispatch must resolve from inferred argument types, and exact effects do not support effect polymorphism until rows exist.

What follows from using it

1. A successful result is a type plus an exact effect set. The result is an elaboration-time fact, not portable image semantics and not a runtime hint.
2. A type mismatch, arity mismatch, effect mismatch, occurs-check failure, unknown word, ambiguous signature, or unsupported expression is a named refusal. It is never a silent default.
3. Lambda parameters are monomorphic inside the body; a `logic` binding may be generalized when its variables are not free in the surrounding environment.
4. There is no implicit numeric or sort conversion, and a multi-result word is not guessed into a tuple function.
5. P4b does not yet attach source ranges or change lowering. Diagnostics name the construct and context; P4c adds source ranges and the lowering handoff.

The type language

```
Type = Variable t
     | Primitive name
     | Sort name
     | Function parameters result effects

Scheme = quantified type variables, type
Constraint = equal type | equal effect set | later, subtype and row constraints
```

`Primitive` is a built-in value class such as `number`, `text`, or `boolean`. `Sort` is a named Scale sort or value set. A function carries its parameter types, result type, and latent effects. A scheme is a type with explicitly quantified variables, used for `logic` and for polymorphic word signatures.

The solver

The solver:

1. applies the current substitution to both sides;
2. aliases a variable to a type only when the occurs-check passes;
3. compares primitives and sorts by equality;
4. compares function parameter counts, then effects, then parameter and result types;
5. solves a list of constraints transactionally, returning the substitution only when every constraint holds.

`generalize` quantifies the free variables of a value type that are not free in the environment. `instantiate` replaces each quantified variable with a fresh variable. `describeType` and `describeError` produce human-readable diagnostics without `%A`, so they remain NativeAOT-safe.

Relation to the term unifier

This is not the D15/D59 unifier. The term unifier operates on image-resident runtime values: atoms, compounds, typed terms, and lambda terms. D59 adds the Miller-pattern fragment of higher-order term unification.

P4 operates on host-side static types. It unifies type constructors and schemes, not applications of unknown functions. It does not use the image store, does not bind runtime variables, and does not implement imitation or projection. Reusing the term unifier here would conflate the term and type namespaces and would silently give type inference runtime semantics it does not have.

Higher-order type unification is only needed if Scale later adds type-level lambdas or higher-kinded variables. That is not in P4a. Ordinary higher-order programming already works through function types, currying, schemes, and instantiation.

Why a normal language runtime cannot do this

The honest statement is not "normal languages cannot infer types." F#, Haskell, ML, and TypeScript all do. The narrower statement is: a normal language's **ordinary runtime and macro layer** cannot perform P4′s open, cross-language, signature-table elaboration without turning into a compiler service or carrying a type IR.

In a normal compiled language, inference is a closed compile-time phase. The inferred type is usually erased, compiler-private, or attached only to compiled metadata. The source declarations and syntax are the input; there is no stable, queryable table that says "this word has these parameter types, this result, and these latent effects, and that table can be replaced by a hot reload." Runtime reflection can see compiled metadata, but not the constraints, schemes, or effects that produced it, and it cannot infer a polymorphic function from them. A compiler API such as FCS or Roslyn can expose the answer, but then the language runtime depends on a particular compiler's private phase and one source language.

Scale changes the placement of the same idea:

1. A word's signature and effects are **image data**, not compiler-private state. `define` consumes that record, and the dispatcher, currying rules, and P4 elaborator all read the same authority.
2. Words are first-class values. A function value has a type with arity, result, and latent effects, so application and partial application are ordinary constraints rather than special syntax.
3. The signature table is part of the live image. A redefinition appends a new record; elaboration can use the current record and the history can explain which one was used.
4. The same contract crosses the package boundary. F# packages expose compiler read-off through FCS; Elixir packages expose their callable contract; Scale does not need a different inference algorithm per host.
5. Diagnostics are Scale values with Scale source positions, not raw host compiler messages leaking through the language.

That is why P4 is possible in Scale in a way it is not normally possible in a host runtime: Scale owns the signature/effect record as language data, while a normal language keeps inference inside a compiler and exposes only the residue. It is an architectural difference, not a different unification algorithm. A normal language could build the same thing, but it would have to move its type information out of the compiler and into a live, queryable language artifact. P4a does not yet read the live image; P4b and P4c connect the core to the expression and signature table.

Staged implementation

1. **P4a, landed:** `components/typing/type-inference.fs` implements the type, scheme, constraint, substitution, occurs-check, unifier, generalizer, instantiator, and diagnostics.
2. **P4b, landed:** `inferExpression` checks `ExpressionAst.Expression` against the internal primitive registry in `components/prelude/evaluator/signatures.fs`: literals, words, calls, lambdas, application, partial application with holes, binding nodes, `if`, blocks, pipelines, and exact effect sets. Eight regression checks cover the documented surface; four registry checks cover primitive types and an inferred lambda.
3. **P4c, partially landed:** source maps, path-aware `InferenceDiagnostic`, and
4. `lowerExpressionWithInference` are implemented; five checks cover path
5. attribution, missing ranges, successful inference, and refusal to lower after
6. a type failure. `parseLocated` and parser-emitted ranges are still open. Effect rows and subtype constraints remain future work.

P4c source ranges and the lowering handoff

P4c keeps the canonical expression tree unchanged. Source locations live in a parallel map, not in `Expression`, so structural equality, the portable JSON, and existing callers do not gain location data.

```
SourceRange = { StartOffset: int; EndOffset: int }   // character offsets, end
exclusive
ExpressionPath = int list                            // root is []; a child appends
its position
ExpressionSourceMap = Map<ExpressionPath, SourceRange>
```

The three source-map types live in `components/core/expression-ast.fs`, shared by the parser and the inference module; `components/typing/type-inference.fs` consumes them and owns the diagnostic shape below. The parser therefore does not depend on the inference module.

```
InferenceDiagnostic =
    { Path: ExpressionPath
      Range: SourceRange option
      Error: InferenceError
      Description: string }
```

A parser that supports P4c emits the expression and its source map together. `parse` keeps returning only the expression; `parseLocated` returns the pair. A missing map entry is a diagnostic with `Range = None`; it is never guessed. Line and column are derived from offsets for display.

Child paths follow the expression constructor's field order. A list field appends its element index after the preceding fields:

1. `Call(target, arguments)`: target `[0]`, argument `i` `[1 + i]`.
2. `Partial(target, arguments)`: target `[0]`, argument `i` `[1 + i]`.
3. `Pipeline(value, stage, _)`: value `[0]`, stage `[1]`.
4. `ReversePipeline(target, value)`: target `[0]`, value `[1]`.
5. `Lambda(_, body)`: body `[0]`; parameters are names, not expression nodes.
6. `Let(_, value, body)`: value `[0]`, body `[1]`.
7. `If(condition, thenBranch, elseBranch)`: `[0]`, `[1]`, `[2]`.
8. `Block(body)` and `Quotation(body)`: body element `i` is `[i]`.
9. `Message(_, fields)`: field `i` is `[i]`.
10. A leaf (`Literal`, `Word`, `Hole`) has no children.

A range ends at the first offset after its last consumed token. Parent and child ranges may overlap; the map is not a tree of disjoint spans.

The author-facing consequences:

1. `scale --prose-typecheck PATH` parses, infers, and lowers without running the program. A type or lowering failure is printed and the exit code is non-zero.
2. A type error names the expression path and, when the parser supplied it, the source range. Unknown words, mismatches, arity, effects, and occurs-check failures all use the same diagnostic shape.
3. Inference failure stops the lowering handoff. The lowering never runs on a failed or partially checked expression.
4. A lowering refusal keeps its existing `LoweringFailure`; a type diagnostic and a lowering refusal are different layers and are never conflated.
5. Existing `lowerExpression` behavior is unchanged. `lowerExpressionWithInference`
6. is the opt-in P4c path and takes the internal registry table and source map.

The contract has explicit limits:

1. P4c ranges cover only expression nodes the parser annotates. `PatternMatch`,
2. declaration nodes, and names that are not expression children are outside the
3. first child-range slice.
4. The portable expression document remains range-free. Source maps are unit
5. metadata; two documents with different ranges remain structurally equal.
6. Pattern matching remains outside P4b/P4c until its own inference contract
7. exists.
8. Effect rows and subtype constraints are not part of P4c; it uses the exact
9. effect sets P4b already checks.

The current implementation supplies precise ranges for call and partial targets and arguments, plus leaf ranges. The other constructor paths above are the contract for the next range slice; until then the parser reports the root range for those nodes.

Boundary

P4 is compile/elaboration-time. Inferred facts are not portable semantics and are not trusted by the kernel. A stale hint is ignored; the checked path remains. Full Huet unification, dependent types, type-level lambdas, and implicit arguments are out of scope.